ENGR 102 Summer Bridge
Day 11 Instructor Key

While Loops, Update Order, Sentinels, and Termination

20 points • approximately 20-25 minutes

Name		UIN		Section	

Boolean-operator convention. Python operators appear as `and`, `or`, and `not`. Their lowercase spelling is required in code.

Show intermediate state for trace credit. Program credit requires one coherent solution. Unless a prompt says otherwise, give exact Python 3 output.

Question 1. Trace update order and termination. — 5 points

```
value = 3
passes = 0
while value <= 20 and passes < 4:
    value = value * 2 - 1
    passes = passes + 1
print(value, passes)
```

Show every after-pass state and the first false condition; then give exact output.

Answer and explanation

After-pass states are (5,1), (9,2), (17,3), and (33,4). The next condition is False because 33 <= 20 is False and 4 < 4 is False. Exact output: 33 4.

Question 2. Repair the update placement. — 3 points

```
number = int(input())
while number != 0:
    if number > 0:
        print(number)
# intended: read the next number each pass
```

Add the one necessary line in the correct location and explain why placing it only inside the if is unsafe.

Answer and explanation

Add number = int(input()) at the end of the loop body, aligned with the if. If it were inside the positive-only branch, a negative nonzero number would skip the update and repeat forever.

Question 3. Program construction — 12 points

Write one complete Python program that satisfies every requirement.

- Read integer amounts until 0 is entered.
- Ignore negative amounts without adding or counting them.
- For positive amounts, compute count and total.
- After 0, print total and count, then print average when count is positive or empty when count is zero.
- Include one immediate-sentinel test.

Answer and explanation

```
amount = int(input())
count = 0
total = 0

while amount != 0:
    if amount > 0:
        count = count + 1
        total = total + amount
    amount = int(input())

print(total)
print(count)
if count > 0:
    print(total / count)
else:
    print("empty")
```

The update is outside the positive branch so negative values cannot trap the loop. The sentinel is excluded because the condition fails before a pass begins.

Immediate 0 prints 0, 0, empty. Inputs 4, -3, 6, 0 print 10, 2, 5.0.

Scoring guidance

2 input/state, 3 loop/update/termination, 2 ignore-negative logic, 2 total/count, 2 safe average/output, 1 immediate-sentinel test.